

Fantasma da Máquina

- **Limite de tempo:** 10 minutos
- **Pontuação baseline:** 28.6
- **Ambiente:** uma GPU (≈ 16 GB de VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 20 MB
- **Armazenamento:** 5 GB
- **Modelos pré-treinados:** apenas [bge-base-en-v1.5](#) — um **encoder** de texto (modelo de embedding).

Tarefa

Coisas estranhas estão acontecendo no Arquivo Nacional do Cazaquistão. Bibliotecários dizem que alguns livros costumavam terminar de maneira diferente, mas ninguém consegue provar isso — todas as cópias são iguais, e todas as histórias ainda fazem sentido. Você foi convidado, na qualidade de pesquisador de IA, para localizar as alterações.



Uma passagem começa como um texto escrito por uma pessoa e, em algum momento, muda silenciosamente para uma continuação gerada por um modelo de linguagem. Lida como um todo, ela parece um texto coerente — mas, em algum lugar no meio, o autor muda de uma pessoa para uma máquina. Sua tarefa é **encontrar essa mudança: o índice do caractere em que a parte humana termina e a parte da máquina começa**.

Cada amostra é uma única string `text`. Há exatamente uma fronteira. Tudo antes dela é humano; tudo a partir dela é gerado por máquina.

Dataset

Passagens em inglês em texto simples, cada uma com uma fronteira.

- **Parte A** (antes da fronteira): um trecho de texto escrito por uma pessoa.
- **Parte B** (a partir da fronteira): uma continuação produzida por um modelo de linguagem, condicionada à Parte A.
- Cada lado tem pelo menos 180 palavras; o comprimento total é de ~500–800 palavras.
- O **boundary_char_index** é o primeiro caractere da **Part B** `text[boundary_char_index:]` é exatamente a parte gerada pela máquina, e `text[:boundary_char_index]` é a parte humana junto com o espaço que separa ambos.

O que você recebe

Você recebe **duas pastas**:

Pasta	Amostras	answers.jsonl?	Use-a para
dataset/train/	1,221	☐ incluído	treinar / fazer fine-tuning do seu método
dataset/test_public/	380	☐ incluído (cópia de desenvolvimento)	executar seu pipeline e calcular sua própria pontuação localmente

No **momento da avaliação**, sua pasta `dataset/test_public/` é **substituída por um conjunto de avaliação oculto**. Ele tem o mesmo formato, mas **sem answers.jsonl**. Seu notebook é executado novamente nele, e o `answers.jsonl` produzido por ele é avaliado.

- O placar público usa um conjunto oculto **test_leaderboard_a** (380 amostras).
- A classificação final usa um conjunto oculto **test_leaderboard_b** (380 amostras).

Todos os três conjuntos de avaliação têm o mesmo tamanho e são extraídos da mesma distribuição que `train`, portanto sua pontuação local em `dataset/test_public/` é uma estimativa razoável da sua pontuação no placar.

Formato em disco

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- Os IDs em `answers.jsonl` correspondem aos IDs em `data.jsonl`.
- `dataset/train/` (com as respostas) está disponível sempre que você treinar ou fizer fine-tuning.

Saída (formato de submissão)

Você deve submeter **um único notebook, que deve se chamar solution.ipynb**. Esse nome de arquivo exato é obrigatório. Qualquer outro será rejeitado sem ser executado.

Seu notebook deve **ler dataset/test_public/data.jsonl** e escrever um único arquivo **answers.jsonl** na raiz do repositório — um objeto JSON por linha, mapeando cada ID de amostra para o índice de caractere previsto

para a fronteira:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` deve ser um **inteiro em $[0, \text{len}(\text{text})]$** .
- Cada ID em `dataset/test_public/data.jsonl` deve aparecer exatamente uma vez. Uma amostra ausente de `answers.jsonl` (ou com um valor não inteiro / fora do intervalo) recebe pontuação 0 para essa amostra.

Pontuação

Para cada amostra, seja p o índice previsto por você e t a fronteira verdadeira. A pontuação por amostra decai exponencialmente com a distância em caracteres:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Isso resulta no seguinte comportamento da pontuação:

- **≈1.0** — caractere exato da fronteira;
- **≈0.78** — desvio de 25 caracteres;
- **≈0.61** — desvio de 50 caracteres;
- **≈0.37** — desvio de 100 caracteres;
- **≈0.01** — desvio de 500 caracteres.

A **pontuação final é a média** das pontuações por amostra entre todas as amostras da divisão (reportada em uma escala de 0–100). A métrica recompensa previsões *próximas*, não apenas exatas.

Restrições

- **Ambiente:** uma GPU (≈16 GB de VRAM), sem internet no momento da avaliação — o modelo permitido (abaixo) já é fornecido. **Limite de tempo: 10 minutos** para a execução completa — isso deve abranger qualquer treinamento / fine-tuning que você realizar no momento da avaliação **mais** a inferência no conjunto de avaliação.
- **Modelo pré-treinado permitido** — esta lista é exaustiva; nenhum outro peso pré-treinado pode ser usado. Ele é **fornecido previamente no ambiente** (carregue-o normalmente, por exemplo, `from_pretrained`; não há internet no momento da avaliação):
 - **bge-base-en-v1.5** — um **encoder** de texto com 110M parâmetros (modelo de embedding). Ele produz embeddings de sentenças/passagens; não é um modelo de linguagem generativo. Você pode usá-lo **como está (features congeladas) ou fazer fine-tuning dele na divisão train** (o fine-tuning completo cabe no orçamento de 16 GB / 10 minutos).
- Ferramentas clássicas / estatísticas não têm restrições: você pode construir qualquer modelo baseado em features (por exemplo, classificadores ou regressores do scikit-learn) sobre features de embedding que você mesmo calcular. *Pesos pré-treinados de aprendizado profundo* estão restritos apenas à lista acima.